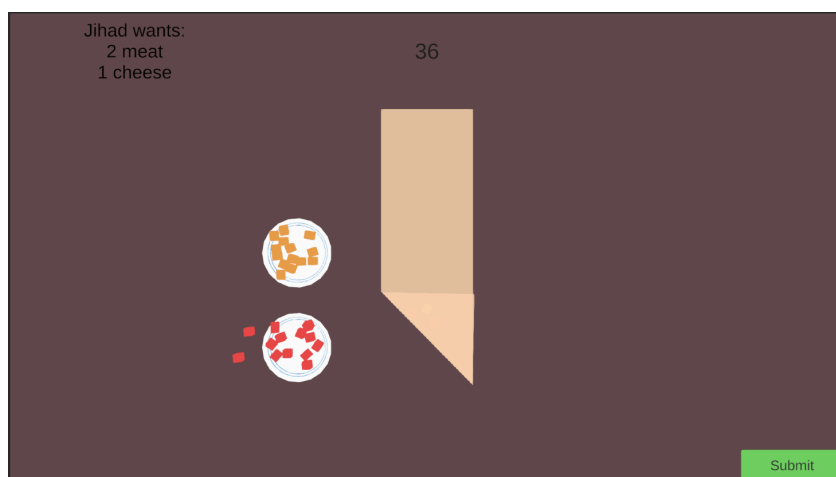


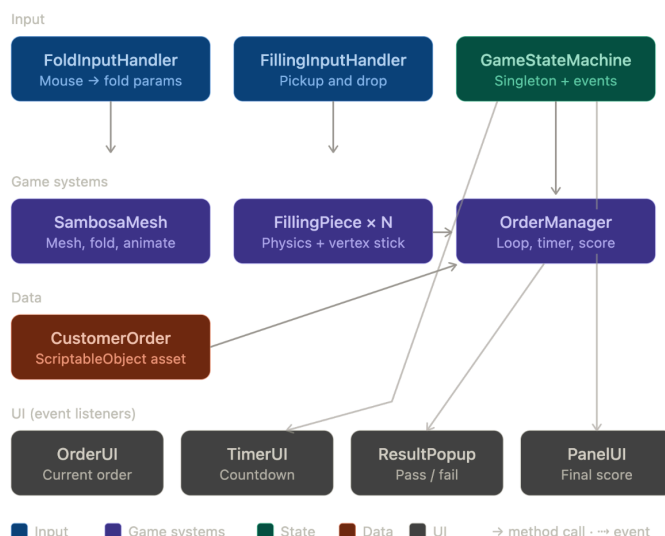
Sambosa Shop: Technical Documentation

A timed mobile prototype that transforms paper-folding mechanics into a sambosa making game: the player folds the dough, fills it with meat or cheese, and serves customers in time. It has you folding dough and inserting the right amount of meat and cheese for each order, serving its customers on time. Unlike most mobile folding games, games that use sprites or baked animations, the dough is a 3D mesh generated procedurally that will bend with the players' drag, and vertex math makes geometry calculation of each fold. The filling is built as physics-based 3D objects that fall under gravity, collide with the dough, and lock onto dough vertices so they bend naturally and follow folds. The user gets their score when time runs out: the customers served correctly out of all customers.



System Overview

The project consists of focused single responsibility scripts, which are communicating through events, a central state machine, and minimal public interfaces. There is no script that references another script's internal data. Input handlers call public methods on SambosaMesh; at the same time, they never touch its vertex arrays.



The main scripts:

- **GameStateMachine**: singleton object state controller with event-based transitions (*OnStateChanged*). Some scripts subscribe and respond to state changes through events, not poll.
- **SambosaMesh**: creating a procedural mesh, doing fold math, smooth vertex animation. Uses a three-array pattern (rest, target, current) and folds animate cleanly as opposed to snapping.
- **FoldInputHandler**: reads mouse input, computes fold parameters from drag direction, and calls *SambosaMesh.ApplyFold()*. Doesn't know how the mesh works internally.
- **FillingPiece**: on hitting the dough, adheres to the closest vertex and folds with it.
- **FillingInputHandler**: hold-to-pickup, release-to-drop. Sets a 'static' flag which the fold handler reads to avoid input conflicts.
- **CustomerOrder**: a ScriptableObject, so new orders can be created from the Unity Inspector without modifications.
- **OrderManager**: powers up the game loop (next customer, submit, reset, end of game), watches the time, and scores submissions.
- **OrderUI, ResultPopup, TimerUI, PanelUI**: four tiny UI scripts respond to one given event (state change, submit result, timer tick, or game over). They are deliberately independent, ensuring that each UI element is replaceable independently of one another.

Scripts talk to each other via events (state changes, submit results) and defined public methods (*ApplyFold*, *SubmitOrder*, *NextCustomer*). Cross-script references are one-directional: input scripts call the mesh and the manager but never the other way around.

Key Design Decisions

There is a three-array vertex pattern in the dough mesh: *baseVerts* (resting), *targetVerts* (where the fold is going), *verts* (what is drawn). Each frame, the *verts* lerp toward *targetVerts*, allowing folds to be animated rather than snapping. One array would result in snapping and no way to compose multiple folds.

Custom fold math instead of Unity Cloth. Cloth would do great for soft body simulation, but competes with manual vertex animation. I applied quaternion rotations over a crease line, full control, no solver to contend with. Cloth migration is planned as future work.

ScriptableObjects for orders. Orders are assets, not hardcoded. New orders can be added from the Inspector without recompiling. Simple C# classes would have worked, but couple data to code only.

Static *IsHolding* flag, a shortcut. To stop fold input from firing during a piece drag, I made use of a static bool. The cleaner version would route both inputs through the state machine. The flag is three lines across two scripts and works reliably for this scope.

Future Work

The biggest upgrade would be to transfer the dough from procedural mesh math into Unity Cloth, which would provide actual soft-body deformation and physical collision with filling pieces, current vertex-sticking is a visual approximation, not real physics. Filling should respawn on the plates too so longer sessions do not deplete ingredients. Even more

importantly, with Cloth in place, the filling could act realistically, pieces would sit inside the folded dough, and an overfilled sambosa would spill its filling out when the dough cannot hold it in. That overflow behavior is the type of mechanic that would allow the game to stand apart from the rest of cooking games.

Setup

Requirements. Unity 6.4 (6000.4.7f1) or later or later. Universal Render Pipeline.

Run. Clone the repository, open the project folder in Unity Hub, then open Assets/Scenes/gameScene.unity and press Play.

Controls. Click and drag an edge of the dough to fold. Hold the mouse button on the filling piece to grab it, move it over the dough, and release to drop. Then click Submit when the sambosa matches the order. Game lasts for 60 seconds, serve as many customers as you can.